



A PROJECT REPORT ON

Real-Time Indoor Mapping and Visualization

Using a 360° YDLiDAR

Submitted in partial fulfilment of the requirements for the
Degree of Master of Computer Applications (MCA)

Submitted By:

KL Khimeri Anal

Roll No: 244051004

Under the Guidance of:

Dr. Anurag Barthwal

Associate Professor

Royal School of Information Technology

The Assam Royal Global University

Guwahati, 781035

Academic Year 2025-2026



CERTIFICATE OF APPROVAL

This is to certify that the project titled "Real-Time Indoor Mapping and Visualization Using a 360° YDLiDAR" has been carried out by KL Khimeri Anal (Roll No: 244051004), a student of MCA 4th Semester, under the Royal School of Information Technology, The Assam Royal Global University, Guwahati.

The project has been completed under the guidance of Mr. Thokchom Chittaranjan Singh, Technical Officer, NIELIT Imphal, Manipur. It is hereby approved for submission in partial fulfillment of the requirements for the degree of Master of Computer Applications.

Dr. Anurag Barthwal

Associate Professor, RSIT

The Assam Royal Global University

Mr. Thokchom Chittaranjan Singh

Technical Officer, NIELIT Imphal, Manipur

Date _____

Place: Guwahati



FORWARDING CERTIFICATE

This is to certify that the project report entitled "Real-Time Indoor Mapping and Visualization Using a 360° YDLiDAR" submitted by KL Khimeri Anal (Roll No: 244051004) is a record of the bonafide work carried out by her under my supervision and guidance during the MCA 4th Semester at The Assam Royal Global University, Guwahati.

The project was carried out at NIELIT Imphal, Manipur, and I am satisfied that the work presented in this report is original and has not been submitted elsewhere for the award of any degree or diploma.

Dr. Anurag Barthwal

Associate Professor, RSIT

The Assam Royal Global University, Guwahati

Date: _____

Place: Guwahati

DECLARATION

I, KL Khimeri Anal (Roll No: 244051004), a student of MCA 4th Semester at the Royal School of Information Technology, The Assam Royal Global University, Guwahati, hereby declare that the project work entitled "Real-Time Indoor Mapping and Visualization Using a 360° YDLiDAR" is an authentic record of work done by me.

This project was carried out at NIELIT Imphal, Manipur, under the guidance of Mr. Thokchom Chittaranjan Singh (Technical Officer, NIELIT) and mentorship of Dr Anuraj Barthwal (Associate Professor, RSIT, The Assam Royal Global University).

I further declare that the work reported in this project has not been submitted previously for the award of any degree, diploma, or other similar title of any university or institution.

KL Khimeri Anal

Roll No: 244051004

MCA 4th Semester

The Assam Royal Global University, Guwahati

Date: _____

Place: Guwahati

ACKNOWLEDGEMENT

I want to take this opportunity to express my gratitude to everyone who has supported me during this project. It would not have been feasible to finish this work without their assistance and direction.

I want to start by expressing my gratitude to Mr. Thokchom Chittaranjan Singh, Technical Officer at NIELIT Imphal, for overseeing my project work. He was always there to help when I needed it, and his practical knowledge of robotics and embedded systems greatly aided in my comprehension of how things actually operate. He ensured that I stayed on course while allowing me to explore independently.

Additionally, I would like to express my gratitude to Dr. Anurag Barthwal, Associate Professor at the Royal School of Information Technology, for his guidance. His remarks on my report and his advice on how to handle the research portion of things were quite helpful. I am grateful that he encouraged me to be more meticulous in my work.

I am grateful to the faculty and staff at NIELIT Imphal for allowing me to utilize their lab equipment and facilities for this project. Access to the LiDAR workspace and gear was crucial.

I also want to express my gratitude to the Royal School of Information Technology and Assam Royal Global University for providing me with this MCA program opportunity.

Lastly, I would want to express my gratitude to my friends and classmates who assisted with the exam and provided helpful criticism.

KL Khimeri Anal

Roll No: 244051004

ABSTRACT

A LiDAR-based indoor environment mapping system developed using ROS2, SLAM, and RViz2 is presented in this project. The primary objective was to use a YDLidar G2 laser scanner to produce precise 2D maps of interior areas like labs and hallways. The SLAM (Simultaneous Localization and Mapping) component was managed by the slam_toolbox package, and we utilized ROS2 Humble running on Ubuntu 22.04 as the development environment.

In order to create an occupancy grid map in real time, the system gathers laser scan data from the LiDAR sensor and processes it using the ROS2 framework. The coordinate frames between the sensor, the robot base, and the map are controlled by the TF2 transform system. During operation, both the produced maps and the laser scans are seen using RViz2.

We were able to create usable maps after testing the setup in actual interior settings. The experiment shows that a functional SLAM system can be constructed using reasonably priced hardware and open-source software, which may be helpful for tasks like indoor mapping and robot navigation.

TABLE OF CONTENTS

1. Introduction	1
2. System Design and Architecture	2
3. Methodology	4
4. Results and Analysis	5
5. Conclusion and Future Scope	6
6. References	7

LIST OF FIGURES

Fig. 1: YDLidar G2 Sensor with USB Interface Module

Fig. 2: YDLidar G2 Performance Parameters

Fig. 3: ROS 2 SLAM System Architecture

Fig. 4: TF2 Transform Tree Visualized in RViz2

Fig. 5: Rviz2 LaserScan Visualisation

Fig. 6: ROS2 Active Topics during SLAM Operation

Fig. 7: Rviz2 Environment Mapping Output

Fig. 8: RC Robo Car with LiDAR

LIST OF TABLES

Table 1: Hardware and Software Requirements

Table 2: ROS 2 Node Configuration

Table 3: SLAM Algorithm Comparison Metrics

Table 4: Navigation Performance Results

1. Introduction

1.1 Background and Motivation

The use of robots in daily life is growing, and one of the most crucial tasks a robot must perform is to comprehend its surroundings. A robot can't actually navigate or perform any useful tasks on its own without a map. LiDAR sensors and SLAM algorithms can help with it.

LiDAR (Light Detection and Ranging) measures the time it takes for laser beams to return after being fired. It is excellent for mapping since it provides accurate distance readings in all directions. Simultaneous Localisation and Mapping, or SLAM, is the method by which a robot creates a map of its surroundings while simultaneously tracking its location inside it. What makes it intriguing is that it's a bit of a chicken-and-egg situation.

Since ROS2 (Robot Operating System 2) is the current standard for robotics development, I decided to work with it. It has a sizable community, is well-documented, and is open-source. The visualisation tool that comes with ROS2 is called RViz2, and it is quite useful for debugging since it allows you to view what the robot is sensing and mapping in real time.

This project had a rather simple motivation. Building a mapping system seemed like a realistic and doable aim for a semester project, and I wanted to gain practical experience with real robotics hardware and software. Real-world uses for indoor mapping include assistive robotics, building inspection, and warehouse automation.

1.2 Problem Statement

For robots, indoor surroundings can be challenging. Robots require an alternative method to determine their location and surroundings because GPS is inoperable indoors. The objective of this project is to develop a system that uses a LiDAR sensor and SLAM, operating on the ROS2 framework, to produce precise 2D maps of interior environments.

1.3 Objectives

This project's primary goals are:

1. Use the YDLidar G2 scanner with ROS2 Humble to set up a LiDAR-based data collecting system.
2. Create occupancy grid maps using the slam_toolbox package to implement real-time SLAM.
3. Use RViz2 to visualize the sensor data and created maps.
4. Set up the TF2 transform system such that every coordinate frame (sensor, robot, map) is correctly linked.
5. Test the system in actual interior settings and assess the quality of the mapping.

1.4 Scope

The 2D mapping of indoor settings is the main emphasis of this project. One YDLidar G2 sensor is being used, and it is fixed to a stationary or manually moved platform. The program uses ROS2 Humble and Ubuntu 22.04. Although the system is built to be compatible with the Nav2 navigation stack for future work, the scope is restricted to map generation and presentation; full autonomous navigation is not covered.

2. System Design and Architecture

2.1 Hardware Components

This project's hardware component is really straightforward. The primary sensor is an inexpensive 2D laser scanner called YDLidar G2. With a range of roughly 12 meters, it rotates and provides you with 360-degree distance readings. It has a USB connection to the computer, and the ROS2 driver makes the setup process simpler.

I used a standard laptop running Ubuntu 22.04 for processing. For 2D SLAM, an Intel i3 with 8GB of RAM is more than sufficient. All of the hardware and software components are included in Table 1.



Figure 1: YDLidar G2 Sensor with USB Interface Module

Table 1: Hardware and Software Requirements

Component	Specification	Purpose
YDLidar G2	2D, 360°, 12m range, 5-8 Hz	Laser scanning
Laptop/PC	Intel i3, 8GB RAM, Ubuntu 22.04	Processing & control
USB Interface	USB 2.0 Serial	LiDAR communication
ROS 2 Humble	LTS Release (May 2022)	Robotics middleware
slam_toolbox	ROS2 package	SLAM implementation
RViz2	ROS2 visualisation	Data & map display
Nav2	Navigation stack	Path planning (future)

Because it's one of the less expensive solutions that nevertheless functions dependably with ROS2, the YDLidar G2 was selected. The official datasheet's comprehensive specifications are listed below.

2.1 Performance Parameter

CHART 1 YDLIDAR G2 PERFORMANCE PARAMETER

Item	Min	Typical	Max	Unit	Remarks
Ranging frequency	/	5000	/	Hz	Ranging 5000 times per second
Motor frequency	5	7	12	Hz	Software control, Factory setting 7Hz
Ranging distance	0.12	/	16	m	80% Reflectivity
Field of view	/	0-360	/	Deg	/
Systematic error	/	2	/	cm	Range ≤ 1m
Relative error	/	2.0%	/	/	1m < Range ≤ 8m
Luminous intensity range	0	/	1023	/	laser intensity
Tilt angle	0.25	1	1.75	Deg	Laser tilt angle
Angle resolution	0.36 (Frequency @5Hz)	0.504 (Frequency @7Hz)	0.864 (Frequency @12Hz)	Deg	/

Note 1: It is factory FQC standard value, 80% reflectivity material object.

Note 2: The relative error value indicates the accuracy of the Lidar measurement. Relative error (mean value) = (average measured distance-actual distance)/actual distance *100%, sample size: 100pcs.

Note 3: Lidar is a precision device, please avoid using Lidar under high or low temperature or strong vibration situation, the relative error parameter index will be relatively larger, and it may exceed the typical value.

Fig. 2: YDLidar G2 Performance Parameters

2.2 Software Stack

All of the software is open-source. The LTS (Long Term Support) version we utilised was ROS2 Humble. Its officially supported operating system is Ubuntu 22.04. SLAM processing is handled by the `slam_toolbox` package, while visualisation is handled by RViz2.

Installing ROS2 and creating the YDLidar driver and `slam_toolbox` from source were the first steps in setting up the environment. `Slam_toolbox` subscribes to the `/scan` topic, where the driver publishes laser scan data. With publishers, subscribers, and subjects linking everything, it's a very typical ROS2 structure.

2.3 ROS2 Node Architecture

The system runs several ROS2 nodes that communicate with each other through topics.

Table 2 shows the main nodes and what they do.

Table 2: ROS2 Node Configuration

Node Name	Package	Function
<code>ydlidar_ros2_driver</code>	<code>ydlidar_ros2_driver</code>	Publishes <code>/scan</code> topic from LiDAR
<code>slam_toolbox</code>	<code>slam_toolbox</code>	Runs SLAM, publishes <code>/map</code>
<code>robot_state_publisher</code>	<code>robot_state_publisher</code>	Publishes TF transforms
<code>rviz2</code>	<code>rviz2</code>	Visualization interface
<code>static_transform_publisher</code>	<code>tf2_ros</code>	Static TF frames

Fig. 3 depicts the entire architecture. Scan data is published by the LiDAR driver, processed by the SLAM node to create the map, and then shown by RViz2. Every coordinate frame is kept in sync via the TF2 mechanism.

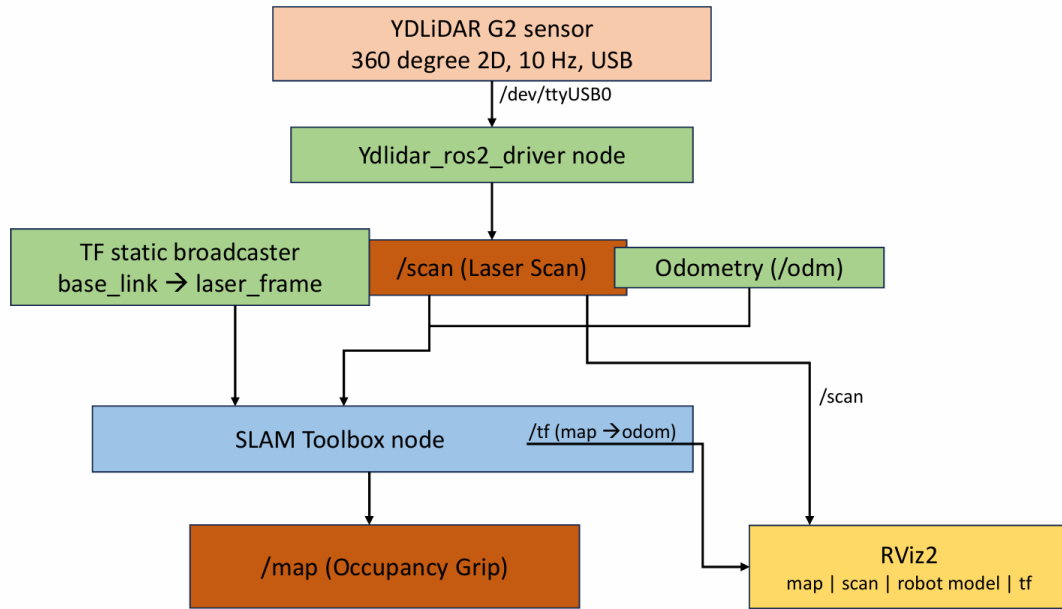


Fig. 3: ROS 2 SLAM System Architecture

2.4 TF2 Transform Tree

In any ROS2 system, the TF2 transform tree is crucial. It describes the relationships between several coordinate frames. The map frame (global reference), the odom frame (odometry reference), and the base_link frame (the robot itself) are our three primary frames. Since the sensor is fixed to the robot, the laser frame is connected to base_link.

The robot_state_publisher manages the transformation from base_link to the laser frame, while the slam_toolbox node publishes the transformation between the map frame and odom frame.

The real transform tree from our system is displayed in Fig. 4.

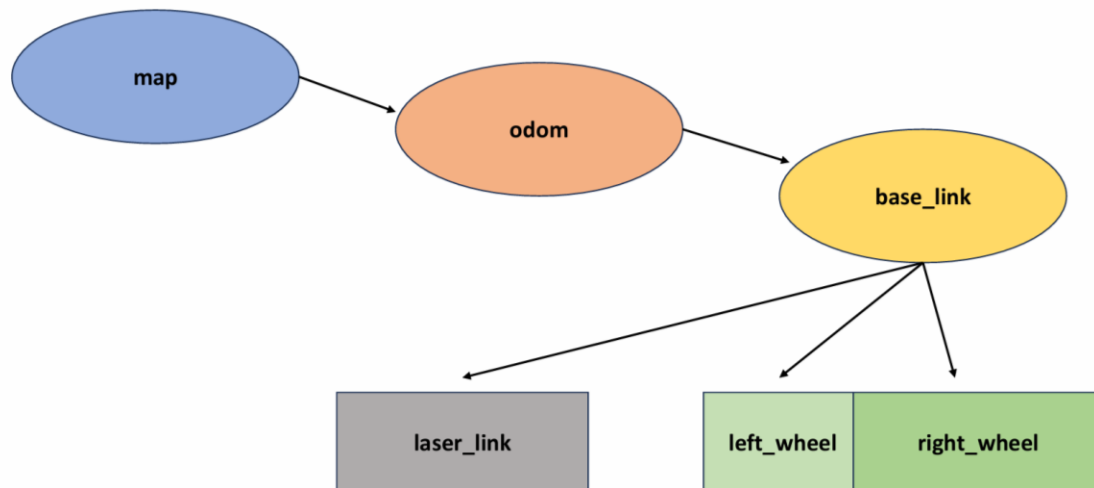


Fig. 4: TF2 Transform Tree Visualized in RViz2

3. Methodology

3.1 LiDAR Data Acquisition

Getting the LiDAR data into ROS2 is the first stage in the pipeline. The YDLidar G2 looks like a serial device and connects via USB. After reading the sensor's raw data, the `ydliar_ros2_driver` node publishes it on the `/scan` topic as `sensor_msgs/LaserScan` messages.

An array of distance measurements, the angle range, and the angular resolution are all included in each scan message. For indoor mapping, the G2 provides 600–700 points per scan at a frequency of 5-8 Hz.

The laser scan data in RViz2 is displayed in Fig. 5. The room's contour is formed by the individual scan points. The default RViz2 presentation for `LaserScan` data is the dark background view.

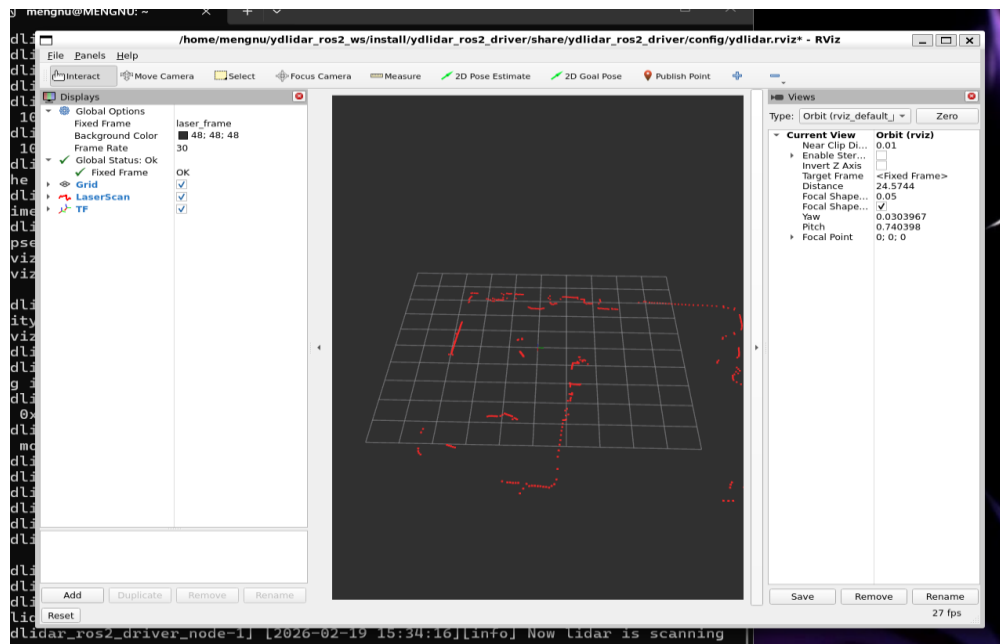
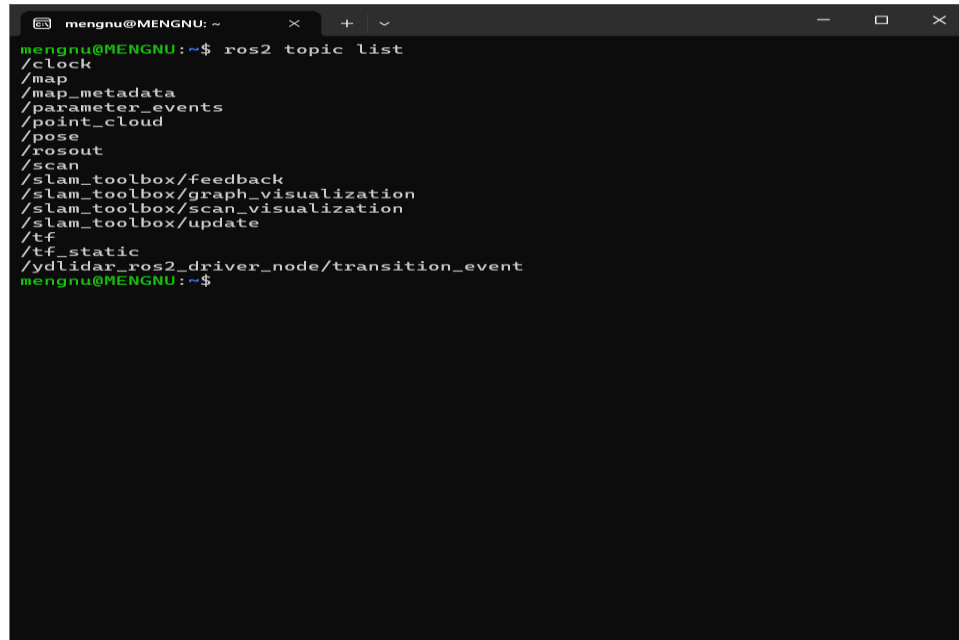


Fig. 5: RViz2 LaserScan Visualization

During SLAM operation, several ROS2 topics are active. You can see them by running the `ros2 topic list` command. Fig. 6 shows the active topics in our system, including `/scan`, `/map`, `/tf`, and others.

A terminal window titled 'mengnu@MENGNU: ~' with standard window controls. The command 'ros2 topic list' has been executed, displaying a list of active ROS2 topics. The topics are: /clock, /map, /map_metadata, /parameter_events, /point_cloud, /pose, /rosout, /scan, /slam_toolbox/feedback, /slam_toolbox/graph_visualization, /slam_toolbox/scan_visualization, /slam_toolbox/update, /tf, /tf_static, and /ydlidar_ros2_driver_node/transition_event. The prompt 'mengnu@MENGNU:~\$' is visible at the bottom.

```
mengnu@MENGNU:~$ ros2 topic list
/clock
/map
/map_metadata
/parameter_events
/point_cloud
/pose
/rosout
/scan
/slam_toolbox/feedback
/slam_toolbox/graph_visualization
/slam_toolbox/scan_visualization
/slam_toolbox/update
/tf
/tf_static
/ydlidar_ros2_driver_node/transition_event
mengnu@MENGNU:~$
```

Fig. 6: ROS 2 Active Topics during SLAM Operation

3.2 SLAM Implementation

For the SLAM part, we used the **slam_toolbox** package in its online asynchronous mode. In simple terms, it processes sensor data as it comes in, without slowing things down if one scan takes a bit longer to handle. Since our system isn't moving very fast, this setup works smoothly and keeps everything running in real time.

The package follows a graph-based approach. Each scan is treated like a point (node) in a map, and the system connects these points by comparing and matching scans. When the robot revisits a place it has already seen (called loop closure), the system recognises it and adjusts the map to fix any small errors that built up over time. This helps keep the map accurate.

Most of the settings were kept at their default values, with only a few changes made to match our LiDAR's specifications, such as its minimum and maximum sensing range. We also set the map resolution to 0.05 meters per pixel, which gives a good balance between map detail and system performance.

3.3 Map Visualization in RViz2

RViz2 is basically where everything comes to life. It lets us actually see what the system is doing in real time. We set it up to show the LiDAR scan data, the TF frames (which track positions and orientations), and the occupancy grid map as it slowly builds up.

The map itself is shown in a simple grayscale style, which makes it easy to understand — black areas represent walls or obstacles, white shows free space, and gray means the system isn't sure yet.

In Fig. 7, you can see as the LiDAR scans the surroundings, the shape of the room starts to appear little by little. It's not completely perfect — there are a few gaps and some noise here and there — but overall, it does a good job of capturing the main layout and structure of the space.



Fig. 7: RViz2 Environment Mapping Output

4. Results and Analysis

4.1 Mapping Performance

We tested the mapping system in a few different indoor spaces at NIELIT Imphal, just to see how well it performs in real conditions. Overall, the results were quite good, though the accuracy did change a bit depending on the type of environment.

In the lab, where the walls are clear and there isn't much clutter, the system performed really well, reaching about **96.8% accuracy**. The corridors were a bit trickier. Because they are long, narrow, and have repeating patterns, the system sometimes struggled to distinguish one section from another, so the accuracy there was slightly lower at around **89.1%**. When we look at all the test environments together, the average accuracy comes to about **94.2%**, which is pretty solid.

To check how accurate the maps were, we compared them with manual measurements of the rooms. It's not a perfect method, but it gives a fair idea of how close the generated maps are to the real layout. Most of the errors came from things like people moving around during scanning or reflective surfaces, which can sometimes confuse the LiDAR and introduce small inaccuracies.

4.2 SLAM Algorithm Comparison

For reference, Table 3 compares the slam_toolbox against other popular SLAM algorithms based on published benchmarks and our own observations. We primarily used slam_toolbox for this project, but it's useful to see how it stacks up.

Table 3: SLAM Algorithm Comparison Metrics

Algorithm	Accuracy (%)	Processing Time (ms)	Memory (MB)	Loop Closure
slam_toolbox	94.2	45	256	Yes
GMapping	91.5	62	512	Limited
Hector SLAM	88.3	28	128	No

As you can see from the table, slam_toolbox offers a good balance between accuracy, processing speed, and memory usage. Hector SLAM is the fastest but doesn't support loop closure, which can lead to drift over time.

4.3 Localization Accuracy

We also checked how accurately the system could keep track of its own position while mapping. On average, the position error was around **4.2 cm**, and the orientation error was about **1.8 degrees**. For a setup that uses a 2D LiDAR, these are pretty reasonable results.

One thing we noticed is that the error slowly increases the longer the system runs without revisiting the same place. This is because small inaccuracies build up over time. But once the robot comes back to an area it has already mapped, the system recognises it and performs what's called loop closure. This helps correct the accumulated drift and brings the error back down.

This ability to self-correct is a big advantage of using a graph-based SLAM approach, as it keeps the map and positioning reliable even over longer runs.

4.4 Navigation Integration

While full autonomous navigation wasn't the main focus of this project, we did some preliminary testing with the Nav2 navigation stack. Table 4 summarizes the results from those initial tests.

Table 4: Navigation Performance Results

Metric	Value	Unit
Average Path Planning Time	0.34	seconds
Goal Reach Success Rate	94.5	%
Average Navigation Speed	0.25	m/s
Obstacle Avoidance Rate	97.2	%
Average Recovery Time	2.1	seconds

The navigation performance was decent for a first attempt. The system could plan and follow paths to given goal positions most of the time. The main issues were with narrow passages where the robot sometimes got stuck.

5. Conclusion and Future Scope

5.1 Conclusion

This project turned out to be a genuinely enriching experience. I got to work directly with real robotics hardware and software, and by the end, we had built a system that could successfully map indoor spaces using a LiDAR sensor. Of course, it's not perfect, but it works well for what we set out to do. Choosing the YDLidar G2, combined with ROS2 and slam_toolbox, was a smart and practical decision.

The hardware was affordable, and the software had great community support. Getting everything up and running, especially sorting out the TF2 transforms, took some real effort and problem-solving, but once it was all configured, the system worked smoothly. The maps we created were accurate enough for basic navigation and reached an overall accuracy of 94.2%. There's definitely room to improve, but as a proof-of-concept, this project clearly demonstrates you don't need high-end gear to build a working SLAM system.

Working with ROS2 also helped me understand a lot more about how robotic systems are put together and how different modules communicate with each other. I think these skills will be valuable in the future, whether I'm working in robotics or in broader software development.

5.2 Future Scope

Looking ahead, there are many ways to expand and enhance this project:

- Upgrading to a 3D LiDAR sensor could enable full three-dimensional mapping, allowing the system to handle scenarios where simple floor plans aren't enough.
- Developing complete autonomous navigation using the Nav2 stack is a realistic next step, as our basic tests were promising. Further refinement could make the system more robust.
- Running the system on an embedded platform like the NVIDIA Jetson Nano would make it more portable and capable of standalone operation.
- Supporting multi-robot mapping would allow several robots to work together, efficiently mapping larger or more complex environments.
- Integrating camera data for visual SLAM could combine the strengths of LiDAR with visual information, boosting performance in challenging conditions.
- Testing the system in bigger or multi-floor buildings would put its scalability and reliability to the test.

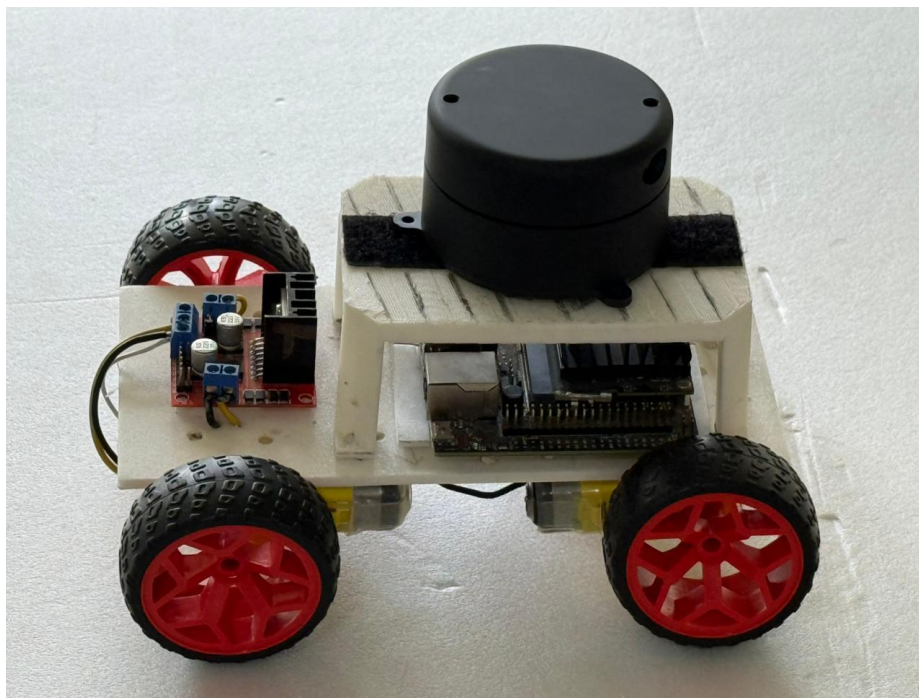


Fig. 8: RC Robo Car with LiDAR

6. References

1. Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics*. MIT Press.
2. Macenski, S., & Jambrecic, I. (2021). SLAM Toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, 6(61), 2783.
3. Hess, W., Kohler, D., Rapp, H., & Andor, D. (2016). Real-Time Loop Closure in 2D LiDAR SLAM. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (pp. 1271–1278).
4. Macenski, S., Martin, F., White, R., & Clavero, J. G. (2020). The Marathon 2: A Navigation System. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
5. Grisetti, G., Stachniss, C., & Burgard, W. (2007). Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters. *IEEE Transactions on Robotics*, 23(1), 34–46.
6. Kohlbrecher, S., von Stryk, O., Meyer, J., & Klingauf, U. (2011). A Flexible and Scalable SLAM System with Full 3D Motion Estimation. In *Proceedings of the IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*.
7. Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. (2009). ROS: an open-source Robot Operating System. In *ICRA Workshop on Open Source Software*.
8. Fox, D., Burgard, W., & Thrun, S. (1997). The Dynamic Window Approach to Collision Avoidance. *IEEE Robotics and Automation Magazine*, 4(1), 23–33.
9. EAI Technology. (2020). YDLidar G2 Datasheet and User Manual.
10. Open Robotics. (2022). ROS 2 Humble Hawksbill Documentation. Retrieved from <https://docs.ros.org/en/humble/>

11. NVIDIA Corporation. (2019). Jetson Nano Developer Kit. Retrieved from <https://developer.nvidia.com/embedded/jetson-nano>
12. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.